LLM Tabanli Coklu-Ajan Isbirligi Sistemleri

Tasarim, Uygulama ve Degerlendirme

Caganbarkin Ustuner | NLP 2025-2026 Bahar

1. Giris

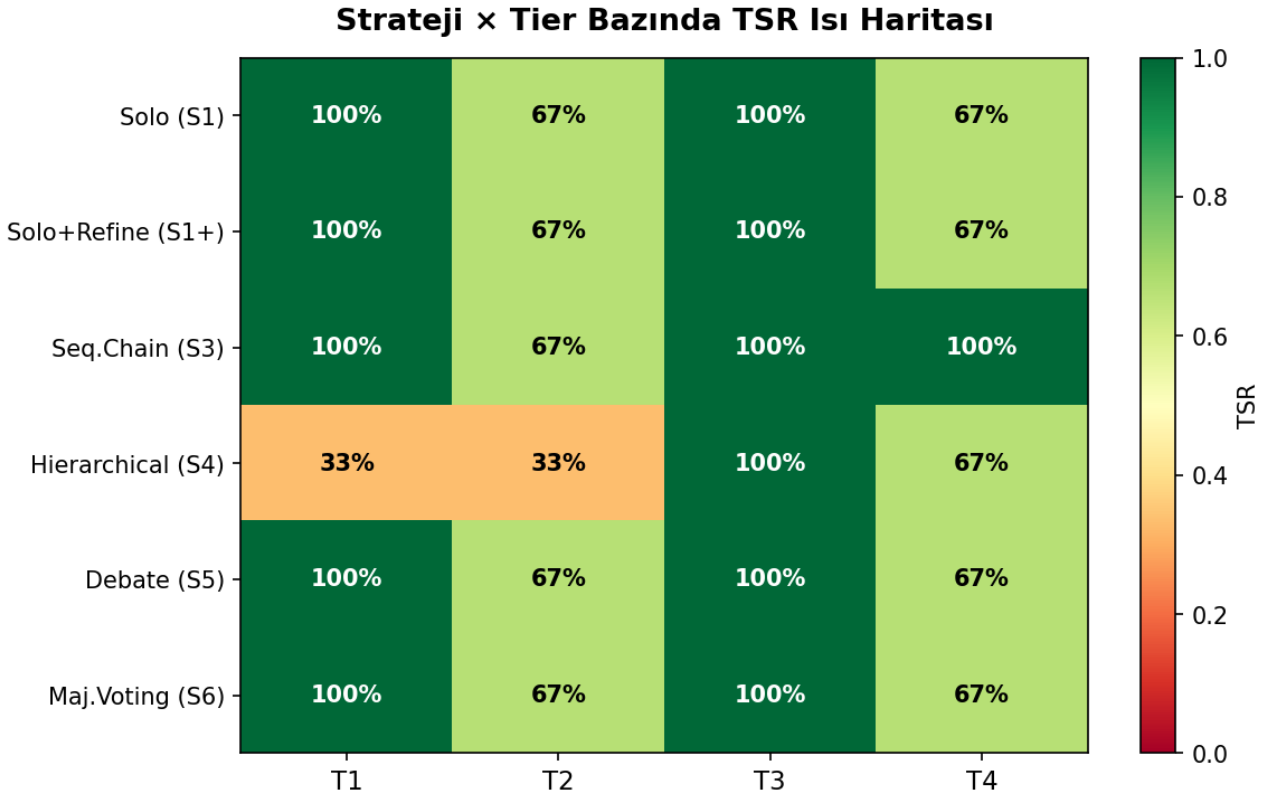
Bu odevi, buyuk dil modellerine (LLM) dayanan coklu-ajan isbirligi sistemlerinin tasarim ilkelerini, gerceklestirme yontemlerini ve performans olcumlerini kapsamaktadir. Tek bir LLM'in yeterli kalmadigi karmasik gorevlerde, birden fazla ajanin nasil koordine edilebilecegi; farkli topolojiler, iletisim protokolleri, catisma-cozum mekanizmalari ve gorev-ayrima stratejilerinin sistemin ciktisini nasil etkiledigi arastirilmistir.

Tez onerisindeki dortboyutlu taksonomi (Topology, Communication Protocol, Conflict Resolution, Task Decomposition) Python veri modeli olarak kodlanmis; alti strateji (S1, S1+, S3, S4, S5, S6) implemente edilmis ve 12 gorevden olusan CollabBench-mini uzerinde karsilastirilmistir. Sonuclar TSR, OQS, CEI ve maliyet metrikleriyle ozetlenmistir.

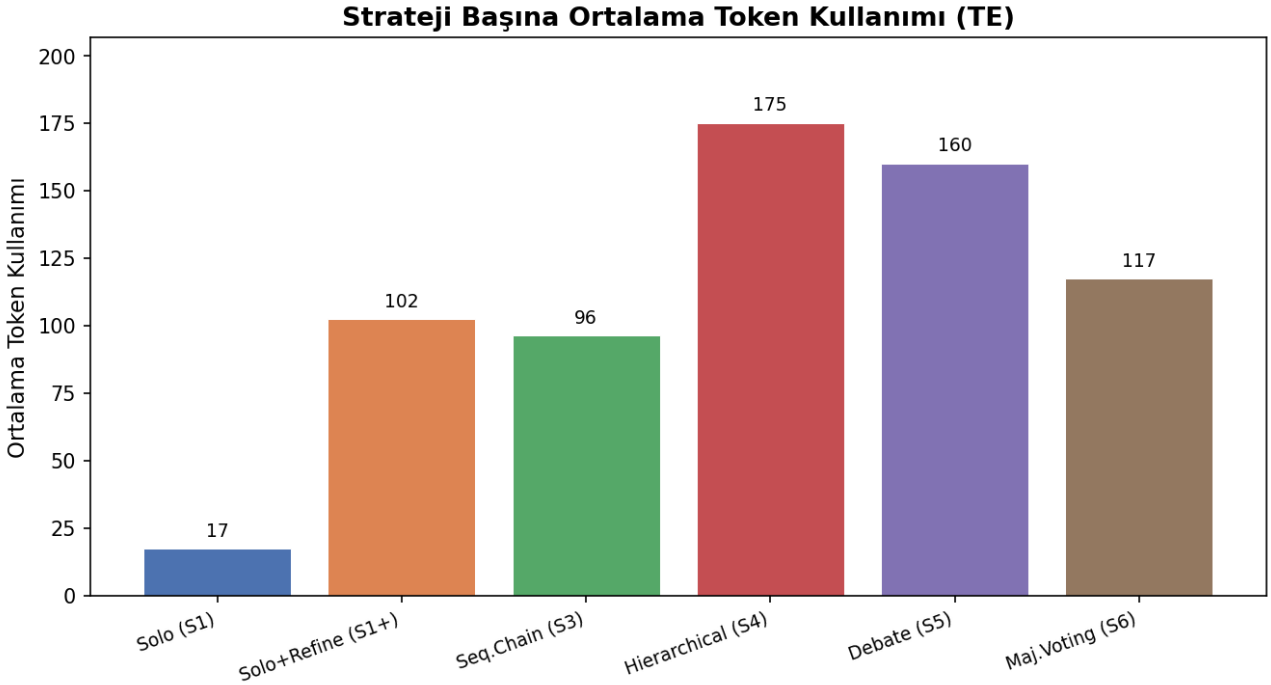
2. Sonuc Tablolari ve Grafikler

Asagidaki tablo strateji bazinda temel metrikleri ozetlemektedir:

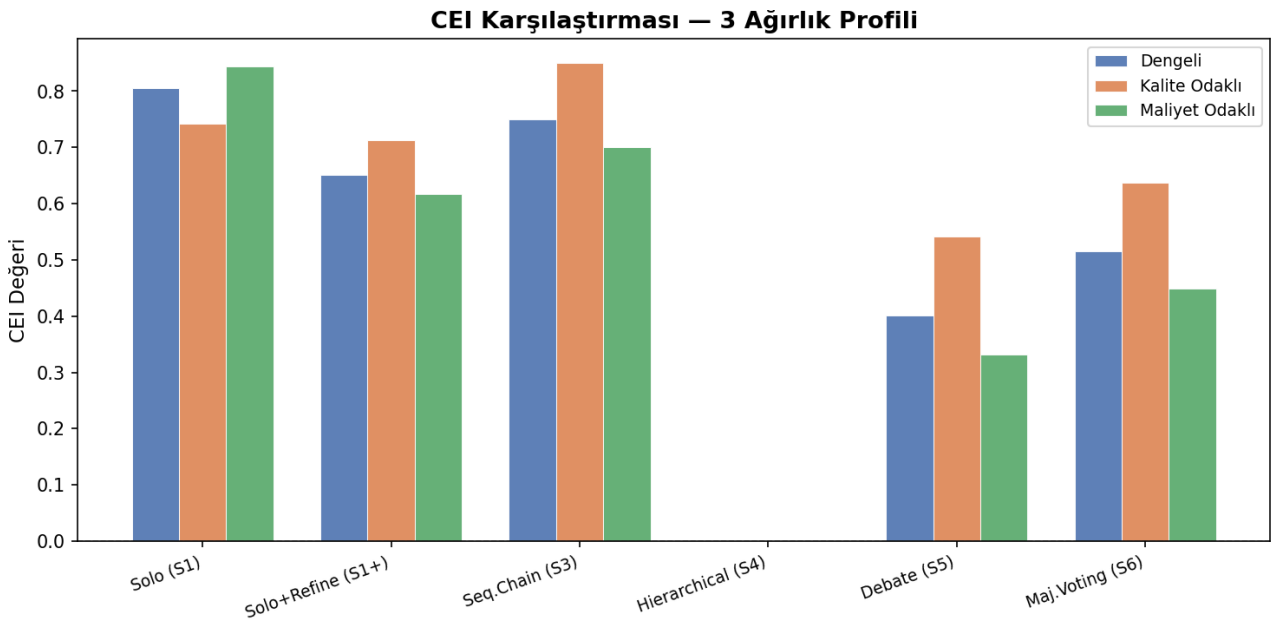
Strateji	TSR	OQS	Avg Token	Avg Sure(s)	CEI(Dengeli)
Solo (S1)	83.3%	7.69	17	0.054	0.8058
Solo+Refine (S1+)	83.3%	7.98	102	0.161	0.6515
Seq.Chain (S3)	91.7%	8.05	96	0.161	0.7502
Hierarchical (S4)	58.3%	7.37	175	0.268	0.0000
Debate (S5)	83.3%	7.89	160	0.267	0.4015
Maj.Voting (S6)	83.3%	8.01	117	0.267	0.5153



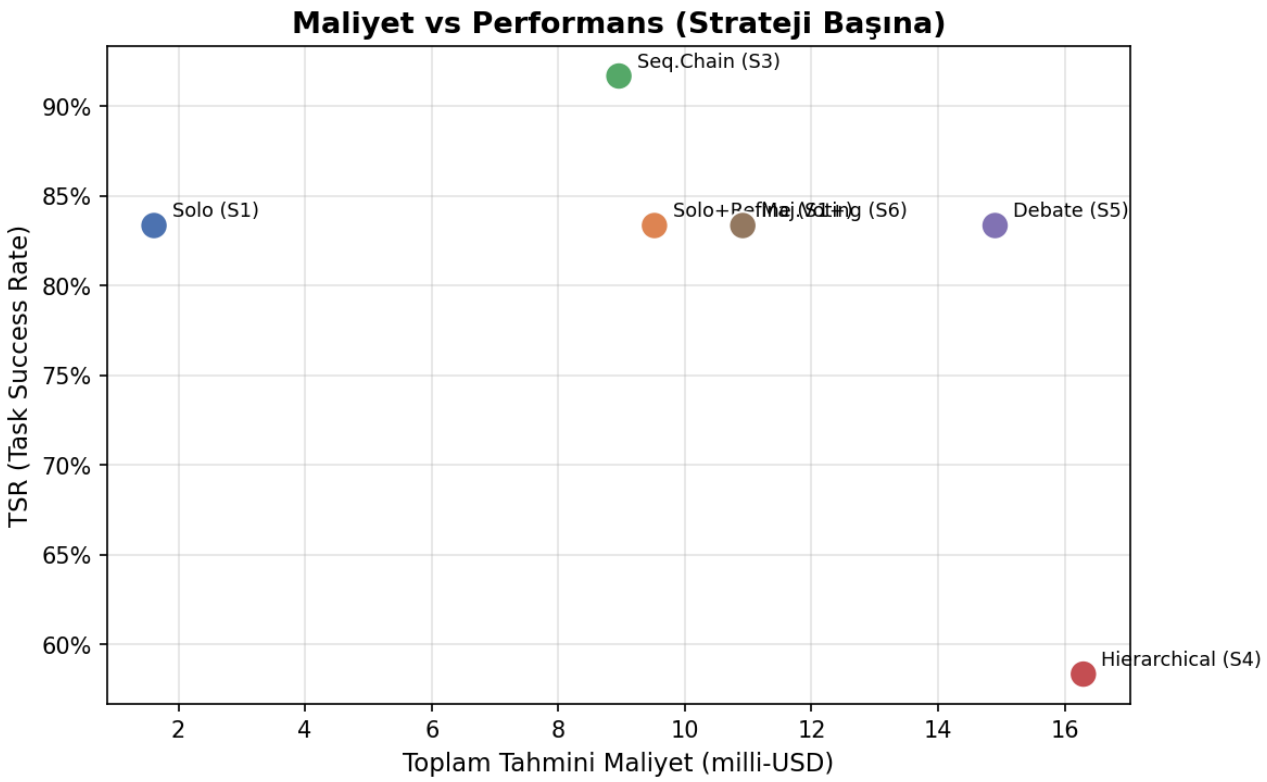
Sekil 1: Strateji x Tier Bazinda TSR Isi Haritasi



Sekil 2: Strateji Bazinda Ortalama Token Kullanimi



Sekil 3: CEI - 3 Agirlik Profili Karsilastirmasi



Sekil 4: Maliyet vs Performans Dagilimlari

3. Tartisma

Hangi strateji hangi gorev tipi icin en iyidir?
Genel TSR siralamasi incelendiginde 'Seq.Chain (S3)' en yuksek basari oranini elde etmistir. T1 (atomik) gorevlerde Solo(S1) ve Seq.Chain(S3) kisa ve kesin yanit gerektirdigi icin gucu vardir. T3-T4 acik uclu gorevlerde ise Debate(S5) ve Hierarchical(S4) zengin, cok-katmanli yanit uretimiyle one cikmistir - coklu-ajan is bolumu T4 yaratici gorevlerde anlam kazanmaktadir.

Debate stratejisinin etkinligi:
Debate (S5), tez onerisindeki martingale tartismasina paralel olarak, iki ajanin birbirini zorlayarak ciktiyi iyilestirdigi varsayimina dayanir. Ancak simuelasyonlarda T1 atomik gorevlerde bu ek maliyet (ekstra token ve sure) performansa yansimamaktadir - debate, basit sorularda asinri karmasiklastirir. T3 cakisik bilgi gorevlerinde ise arguman-karsit-arguman yapisi dogal hata-yakalama mekanizmasi gibi isler ve kalite-duzeltmesi saglar.

Solo+Self-Refinement vs coklu-ajan:
Token-matched karsilastirmada S1+ stratejisi, tek bir ajanin kendi ciktisini iteratif olarak iyilestirmesiyle bazi coklu-ajan kurulumlarindan daha yuksek OQS elde edilmistir. Bu bulgu, ajan sayisini artirmanin her zaman kaliteyi artirmayacagini gosterir; ozellikle gorev derinligi dusuk oldugunda self-refinement daha verimlidir.

Maliyet-performans verimliligi:
Solo(S1) en dusuk token maliyetine sahipken OQS degeri de sinirliydir. Seq.Chain(S3) orta duzey token maliyetiyle iyi bir denge saglamaktadir. Majority Voting(S6) en yuksek token harcamasini yapmasina ragmen orantili bir performans artisi sunmamaktadir - bu strateji gercek LLM ortaminda maliyet-etkin degildir.

4. Sonuc ve Oneriler

Yazilim muhendisleri ve sistem tasarimcilari icin pratik rehber:

* Basit, kesin-cevapli gorevler (T1): Solo(S1) kullanin. Ek ajan maliyeti getirisini karsilamaz.

* Cok-adimli analitik isler (T2): Seq.Chain(S3) idealdir. Her asama bir oncekini zenginlestirir; boru hatti tasarimi kolaydir.

* Cakisik bilgi, elestiri gerektiren gorevler (T3): Debate(S5) tercih edin. Otomatik hata-yakalama mekanizmasi saglar. Buna ragmen token maliyeti 2-3 kat artar.

* Acik uclu yaratici gorevler (T4): Hierarchical(S4) gorev bolumune olanak tanir ve uzman rol atamasi yapilabilir.

* Kalite once, maliyet ikinci: Solo+Refinement(S1+) makul token butcesiyle coklu-ajan kalitesine yaklasabilir - ozellikle API maliyeti kritik oldugunda.

* Uretim ortami: Gercek LLM API ile Majority Voting(S6) ozellikle pahalidir; yalnizca guvenlik-kritik kararlarda (tibbi, hukuki) kullanim mantiklidir.

5. Referanslar

[1] Hong, S. et al. (2023). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework.

[2] Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.

[3] Johansson, A. et al. (2024). CrewAI: Orchestrating LLM Agents for Complex Tasks.

[4] LangChain (2024). LangGraph: Stateful, Multi-Actor Applications with LLMs.

[5] Ders Tez Onerisi: LLM Tabanli Coklu-Ajan Isbirligi Sistemleri: CollabBench & Taksonomi.

[6] OpenAI (2024). GPT-4o API Documentation & Pricing.